CSC3066
Deep Learning
More on Gradient Descent

Anna Jurek-Loughrey

Outline

- What did we learn so far?
  - Neural Networks (weights, bias, activation function, loss function)
  - Perceptron/Multilayer Perceptron
  - Training Perceptron with simple update rule
  - Gradient Descent and Backpropagation for training ANNs
  - Applying Gradient Descent and Backpropagation for training Perceptron
- Coming in the next lectures:
  - More on Gradient Descent: different variants, selecting value of learning rate
  - Different activation functions and error functions that can be used with NNs
  - Applying Gradient Descent and Backpropagation for training Multilayer Perceptron

Outline

- Different variants of Gradient Descent algorithm
- Deciding on the Learning Rate value

Recap on Gradient Descent and Backpropagation

- The process of training a neural network is to determine a set of parameters that minimize the difference between expected value and model output. This is done using gradient descent algorithm.
- Gradient Descent is an optimisation algorithm that aims to minimise a differentiable objective function by iteratively moving in the direction of the steepest descent as defined by the negative of the gradient of the function.

Recap on Gradient Descent and Backpropagation

- Backpropagation is a method which calculates the gradients of the error function with respect to the neural network parameters using chain rule, computing the gradient one layer at a time, iterating backward from the last layer.
- It is commonly used by the gradient descent algorithm to adjust the parameters of the network.

Recap on Gradient Descent and Backpropagation

- Steps of a single iteration of the Gradient Descent algorithm:
  - Pass the input through the Neural Network and calculate the output of the model.
  - Using the output of the model and the target output, calculate the value of the loss function.
  - Calculate the gradient of the loss function with respect to all the network parameters.
  - Update the network parameters by subtracting the corresponding gradient value from their current values, multiplied by a learning rate.

Training Neural Network:

Variants of Gradient Descent Algorithm

Variants of Gradient Descent

- Depending on how much data is processed in a single iteration, there are three different variants of the Gradient Descent algorithm:
  - Batch Gradient Descent
  - Stochastic Gradient Descent
  - Mini-batch Gradient Descent

Variants of Gradient Descent: Batch GD

- Batch Gradient Descent (BGD): all training data is pass through each iteration. The parameters are updated after a full sweep through the training data.
- BDG calculates the error for each example in the training dataset, but only updates the model after all training examples have been evaluated based on the averaged gradients.
- One cycle through the entire training dataset is called a training epoch. Therefore, it is often said that BGD performs model updates at the end of each training epoch.

Variants of Gradient Descent: Batch GD

- Batch Gradient Descent (BGD):

```
for i in range(num_epochs):
    grad = compute_gradient(data, params)
    params = params - learning_rate * grad
```

Variants of Gradient Descent: Batch GD

- Batch Gradient Descent (BGD):
  - The convergence of the weights to the optimal weights is very stable. Using the entire train dataset guarantee a good estimate of the true gradient (no noisy gradient problem).
  - The learning process is slow when the train dataset is large, only one update after all samples have been processed.
  - The algorithm can get stuck in a local minimum of the loss function and never reach the global optimum at which the neural network achieves the best results. This is due to the stable error gradient.

Variants of Gradient Descent: SGD

- Stochastic Gradient Descent (SGD): a single random sample is introduced on each iteration.
- SGD calculates the error and updates the model's parameters after single example from the training dataset has been processed by the network.

```
for i in range(num_epochs):
    np.random.shuffle(data)
    for example in data:
        grad = compute_gradient(example, params)
        params = params – learning_rate * grad
```

Variants of Gradient Descent: SGD

Stochastic Gradient Descent (SGD):

- The learning process is much faster since very little data is manipulated in each iteration. Parameters are being updated more often than in case of BGD.
- It makes it possible to train a Neural Network on huge train set, since only one instance needs to be in memory at each iteration.
- Frequent updates can result in a noisy gradient signal, which may make the error jump around (have higher variance over training epoch) and make it hard for the algorithm to settle on a minimum.
- At the same time, the noisy update process can allow the model to avoid local minima.

Variants of Gradient Descent: Mini-Batch GD

- Mini-Batch GD: offers trade-off between SGD and BGD.
  - It divides the training set into batches of size n and approximates the loss based on each batch.
  - It makes more updates per epoch compared to BGD, which helps to speed up the training process.
  - The loss approximation is not as noisy as with SGD since we are using more training examples. Mini-batch GD can get closer to the minimum than SGD.

Variants of Gradient Descent: Mini-Batch GD

- Mini-Batch GD:

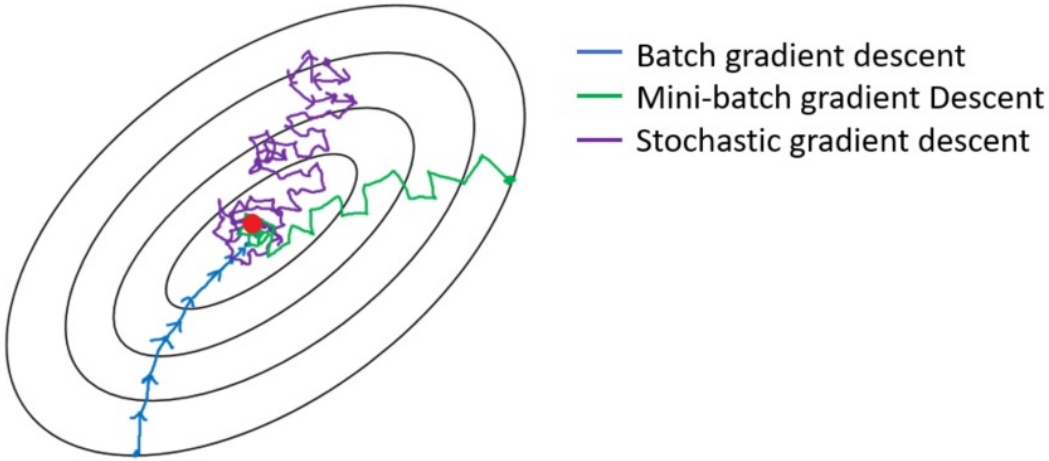
```
for i in range(num_epochs):
    np.random.shuffle(data)
    for batch in radom_minibatches(data, batch_size=32):
        grad = compute_gradient(batch, params)
        params = params – learning_rate * grad
```

Variants of Gradient Descent: Mini-Batch GD

- Mini-Batch GD: selecting the size of mini-batch (hyperparameter)
  - If the dataset is small (less than 2000 examples) then use batch gradient descent
  - For larger datasets, typical mini-batch sizes are: 64, 128, 256, 512, 1024.
  - Mini-batch size must fit in your CPU/GPU memory.

Variants of Gradient Descent: Comparison

- Gradient descent variants' trajectory towards minimum



Neural Network:

Learning Rate

Gradient Descent: Learning Rate

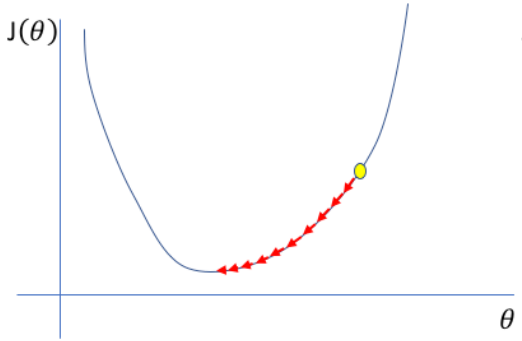
- Learning Rate is a hyperparameter that controls how much the parameters of the neural network are adjusted with respect to the loss gradient.

$$w^{new} = w - \alpha \frac{\partial E}{\partial w}$$

- How much should the parameters be updated in each iteration?

Gradient Descent: Learning Rate

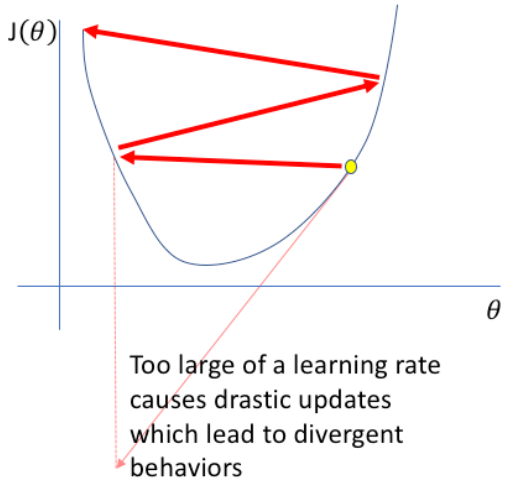
- If the learning rate is too small, gradient descent can be slow to converge.



A small learning rate requires many updates before reaching the minimum point

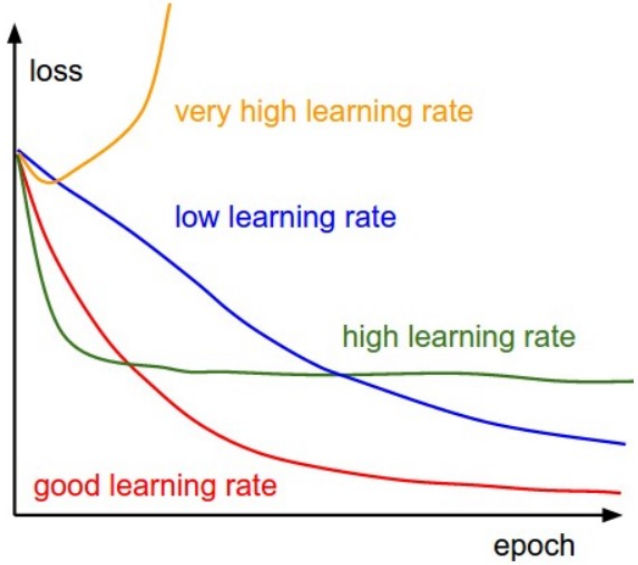
Gradient Descent: Learning Rate

- If the learning rate is too big then the optimisation may be unstable (bouncing around the optimum, loss may not decrease on every iteration), gradient descent may not converge.



Gradient Descent: Learning Rate

- How should we choose the value of learning rate?



Gradient Descent: Learning Rate

- Fixed Learning Rate. We can use a fixed α that is determined by trial and error. A default value of 0.01 typically works for standard multi-layer neural networks with standardized input but we should not rely exclusively on this single value.
- Adaptive Global Learning Rate. Learning rate can be adapted during training based on time (annealing). Start with large α for the first iterations and shrink it for each consecutive iteration.
- Adaptive Local Learning Rate. Learning rate can be adapted for individual parameters of the model (AdaGrad, RMSprop, Adam optimizers) → we will get back to this.

Next to come:

Different Variants of Activation and Error functions